
MemMux: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

Developers increasingly run not one coding agent but a fleet of them side by side on a single workstation. The tools they reach for—terminal multiplexers like `tmux` and a new generation of agent managers—were built to arrange windows, not to govern memory. When ten agents each spawn language servers, test runners, and browsers, no standard tool in that stack can say how much memory belongs to which agent, guarantee that a terminated agent’s descendants are actually gone, notice a child that has escaped its agent, or keep the machine off the swap cliff without an OOM kill silently discarding uncommitted work. We argue these are *runtime-verification* problems: an agent-hosting substrate should continuously emit observable signals that an operator—or an auditor—can monitor and check while agents run in production. We present MEMMUX, a local runtime that treats resource governance as a set of checkable guarantees—honest per-agent attribution, complete reclamation, escaped-process visibility, bounded footprint under overcommit, and low monitoring overhead—and a claims-disciplined benchmark that measures each guarantee against `tmux`, a purpose-built agent multiplexer, and a raw-process baseline running identical workloads. On a 32 GB Linux reference host, as the fleet grows to ten agents, MEMMUX holds peak footprint flat at 435 MiB while the ungoverned tools grow linearly to 2.1 GB; it reclaims 100% of a terminated agent’s process subtree where the raw baseline strands half of it (ten orphaned processes, 303 MiB, at $N=10$); and it is the only system that surfaces an escaped child, detecting 10/10 injected escapes versus a capability the others structurally lack. We report an honest cost: the always-on 1 Hz attribution scan stays near 0.6% CPU for a single agent but reaches 2.7% at ten, over our 2% target. Driving the same harness with real Claude Code sessions confirms that 100% attribution and low overhead carry over from the stub to live, multi-process agent trees. We release the engine, the benchmark, and a one-command reproducer.

1 Introduction

A year ago, running an autonomous coding agent meant babysitting a single session. Today it is normal to fan a task out across several agents at once—one refactoring, one writing tests, one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have improved quickly [4, 10, 12, 13]; the substrate they run on has not. Developers arrange these

35 fleets with terminal multiplexers such as `tmux` [8] or with purpose-built agent managers, and
36 both were designed to place windows on a screen, not to answer questions about memory.

37 Those questions matter because a coding agent is not a single process. It is a tree: the
38 model’s CLI, a shell, language servers, test and build workers, sometimes a headless browser.
39 A handful of such trees can exhaust a laptop’s RAM. When they do, the operating system’s
40 response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is
41 killed mid-write, or the whole machine grinds, and the agent’s uncommitted edits are simply
42 gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does
43 not know that pane 7’s language server belongs to the same logical agent as pane 3’s test
44 runner; it cannot tell you that a double-forked child has escaped supervision; and it has no
45 budget it is trying to keep.

46 We think the right lens for this is *runtime verification* [1]: rather than prove properties of
47 the substrate ahead of time, instrument it so the properties it claims can be *checked while it*
48 *runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous
49 move a decade ago—Borg combined admission control, over-commitment, and process-level
50 isolation, and treated monitoring as a first-class output [11]. We bring that stance down to
51 a single developer machine and to the specific failure modes of agent fleets.

52 This reframes an infrastructure problem as a verification one, on an axis the agent-evaluation
53 literature largely leaves open. Most work on verifying agents asks whether an agent’s *outputs*
54 are trustworthy—did it resolve the issue, is the reward genuine, is the behaviour aligned.
55 We ask a complementary question that only sharpens once agents run in parallel and unat-
56 tended: does the *substrate that hosts them* uphold checkable operational guarantees—honest
57 accounting, complete reclamation, no silently lost work—that an operator can observe while
58 the fleet runs? These are always-on signals over the live execution, not one-shot scores, and
59 a fleet whose host cannot answer them is unverified no matter how capable each agent is.

60 This paper contributes: (i) **five verification signals for agent fleets** (Section 4)—
61 honest per-agent attribution, complete reclamation on teardown, escaped-process visibil-
62 ity, bounded footprint (and preserved work) under overcommit, and bounded monitoring
63 overhead—each a monitor with a stated threshold; (ii) **MemMux** (Section 3), a local run-
64 time that enforces them; and (iii) **a claims-disciplined benchmark** (Section 4) that drives
65 `tmux`, an agent multiplexer, and a raw baseline through *identical* deterministic workloads,
66 tags every process as agent or manager overhead, and never reports a number it did not
67 measure—where a competitor cannot be driven honestly, it is recorded as skipped, not es-
68 timated. On a 32 GB Linux reference host the guarantees hold and, under overcommit,
69 separate MEMMUX from the ungoverned tools by $\sim 5\times$ at ten agents (Section 5); we are
70 candid about the scope—a deterministic-stub workload, modest fleets ($N \leq 10$), and an
71 overhead target not yet met—in Section 6.

72 2 Background and Related Work

73 **Language-model agents.** Interleaving reasoning with tool use [13] and adding self-
74 reflection [10] turned language models into agents that take long sequences of actions. Cod-
75 ing is the canonical proving ground: SWE-bench frames the task as resolving real GitHub
76 issues against a repository [4], and SWE-agent shows a well-designed agent–computer inter-
77 face matters as much as the model [12]. Multi-agent settings, first studied for open-ended
78 behavior [9], are now a practical reality on developer machines. This work is orthogonal to
79 the agents: we govern whatever they spawn.

80 **Terminal multiplexers.** `tmux` [8] is the default way developers run many long-lived ses-
81 sions at once. It is excellent at persistent panes and detach/reattach—the honest baseline
82 any “run many agents” tool must beat—but a pane is just a process group under a server
83 with no notion of a logical agent, a memory budget, or a process that has wandered out of
84 its subtree.

85 **Resource management.** Cluster managers solved fleet-scale governance long ago: Borg
86 with admission control and over-commitment [11], Mesos with two-level scheduling [3], and

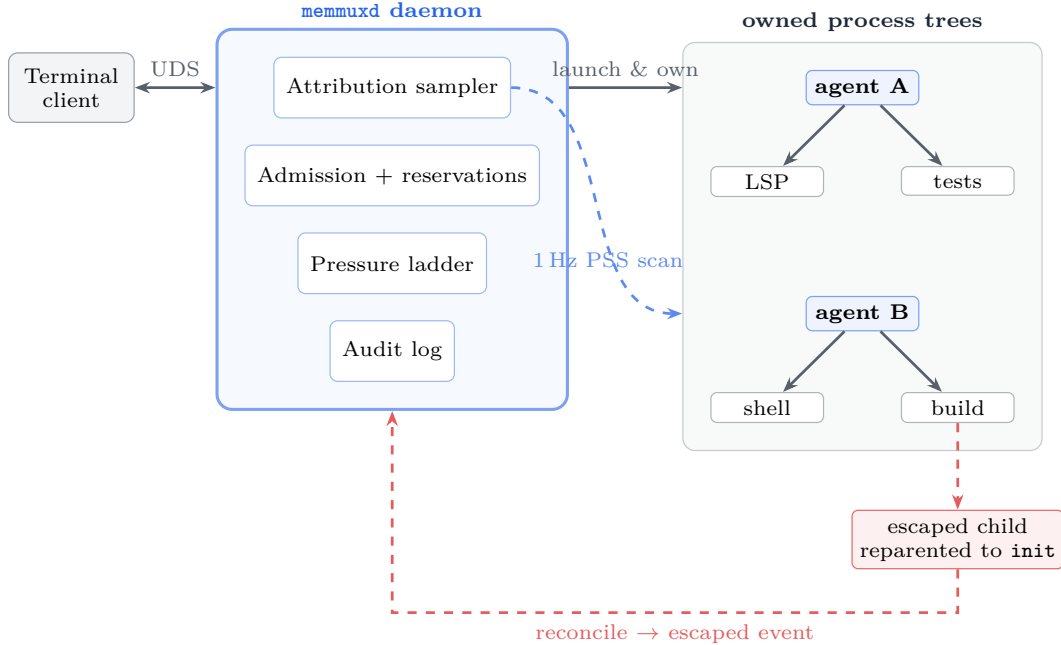


Figure 1: MEMMux architecture. A terminal client drives the `memmuxd` daemon over a Unix domain socket. The daemon launches and *owns* each agent’s process tree; a 1 Hz sampler sums proportional memory (PSS) across every owned subtree (H1), reconciliation flags any child that double-forks out to `init` (H3), and admission, the pressure ladder, and an audit log turn the budget into checkable, logged decisions (H2/H4/H5).

87 Kubernetes as the open-source lineage [2]. The enforcement primitives they use—Linux control groups [5] and the OOM killer [6]—live in every modern kernel. MEMMux borrows the
88 ideas (reserve, admit, reclaim before you thrash) but targets a single un-orchestrated lap-
89 top, and it deliberately avoids hard cgroup containment in favor of signal-based supervision.
90 This is an explicit trade-off: cgroups give hard, kernel-enforced limits but need privilege
91 and per-machine setup, whereas signal-based supervision runs unprivileged and portably
92 (macOS included) at the cost of best-effort rather than hard containment—a child can, for
93 an instant, escape before reconciliation catches it (H3).
94

95 **Memory accounting.** Summing resident set size (RSS) over a process tree double-counts
96 shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup`
97 divides each shared page by its sharers and is the honest per-process figure [7]; macOS offers
98 `phys_footprint` as its analogue. MEMMux attributes by PSS (RSS fallback) so a task’s
99 number reflects memory it is responsible for.

100 **Runtime verification.** Rather than verify a system statically, runtime verification instru-
101 ments it and checks a specification against the execution trace, reacting to violations [1].
102 Our five signals are exactly such monitors: cheap, always-on checks with explicit thresholds
103 whose failures are events, not silence.

104 3 The Design of MemMux

105 MEMMux is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing,
106 and a terminal client that attaches over a Unix domain socket. A *task* is a durable, logical
107 agent; a *runtime* is one physical launch of a provider inside a pseudo-terminal. The design
108 is organized around making each signal checkable (Figure 1).

109 **Attribution.** Once per second the daemon snapshots the host process tree and, for each
 110 running task’s provider root, sums the PSS of the root and all descendants. That per-task
 111 figure feeds the budget, the client’s task view, and—because we record which pids were seen
 112 under each root—the escaped-process check. Attribution is scoped to processes MEMMUX
 113 launched, so unrelated host activity never pollutes a task’s number.

114 **Recursive reclamation.** When a task is terminated (by the operator or because its
 115 provider exited), MEMMUX reaps the *entire* owned subtree. The order is load-bearing: the
 116 subtree is signalled while the root is alive, because once the root dies its children reparent to
 117 `init` and the tree that identifies them is lost. For a provider that already exited, MEMMUX
 118 reaps from the pids recorded while the task ran. The target is $\geq 99.5\%$ of owned descendants
 119 gone within ten seconds.

120 **Escaped-process reconciliation.** A child that double-forks and reparents to `init` has
 121 escaped its agent—the classic way a leak hides. Each sample, MEMMUX reconciles the
 122 pids seen under a task against the live tree; a once-owned pid now living outside every task
 123 subtree is flagged *escaped* and surfaced as an event. Reconciliation is scoped to the managed
 124 set, so the signal is a real “we lost track of a process we started,” not host noise.

125 **Admission and the pressure ladder.** The daemon holds a memory budget (physical
 126 RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting
 127 a task reserves its predicted peak; if the reservation would exceed the budget the task
 128 stays queued with a visible reason rather than over-subscribing the machine. When live
 129 usage climbs anyway, a graduated *pressure ladder* responds before the machine thrashes:
 130 reclaim escaped children, hibernate the lowest-priority idle task, recycle the largest provider,
 131 and—only at the top, and only after persisting a checkpoint of the victim’s Git state—
 132 terminate the lowest-priority tree. Preserving work always precedes destroying a process.
 133 Every governance action writes an audit decision with its reason and evidence, which is
 134 what makes the system verifiable rather than merely automatic.

135 4 Verification Signals and Methodology

136 We frame governance as runtime monitors over the fleet’s execution. The observed *trace*
 137 is the sequence of once-per-second process-tree samples and lifecycle events; each signal
 138 is an *invariant* over that trace with an explicit threshold (Table 1), and a breach is sur-
 139 faced as a violation event rather than silently tolerated. Four are properties of the running
 140 system (H1–H4); H5 is the *cost* of monitoring, not an invariant, and H4 bundles three sub-
 141 properties—admission, bounded footprint, and preservation of uncommitted work—which
 142 we report separately below. MEMMUX both *observes* these signals and *enforces* them (ad-
 143 mission, reclamation); the baselines can only be observed. These are empirical monitors
 144 with measured pass/fail behaviour, not machine-checked proofs, so we claim the signals are
 145 checkable and checked, not that the engine is sound and complete in the formal sense. A
 146 benchmark harness drives every launcher through identical, deterministic workloads and
 147 evaluates the monitors from one external process sampler—an *oracle* using the daemon’s
 148 own accounting code, pointed at whatever root pids a launcher reports, so measurement is
 149 product-agnostic.

150 **Launchers.** Each launcher starts N identical agents and reports the pids that constitute
 151 the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated
 152 server), and MEMMUX (create N tasks through the real daemon). A fourth, the `herdr` agent
 153 multiplexer, is driven through its socket API where installed; on our stock Linux reference
 154 host it was absent and is reported as skipped rather than guessed, as are `dmux` (which
 155 requires a controlling terminal and cannot be driven headlessly), `cmux`, and `agentmux`.

156 **Workload.** A deterministic stub agent replays a recording of `allocate` / `spawn-child` / `emit`
 157 / `sleep` steps, touching every page it allocates so its resident memory is real. Using a stub
 158 rather than a live model is a deliberate choice: it makes runs reproducible and identical
 159 across launchers, at the cost of realism (Section 6). A *hold* scenario keeps N agents, each

	Signal	Monitor / threshold
H1	Attribution	$\geq 95\%$ of a launched tree’s proportional memory mapped to its owning task
H2	Cleanup	$\geq 99.5\%$ of an owned subtree gone ≤ 10 s after the launcher’s normal teardown
H3	Escaped visibility	every process that escapes its agent is surfaced as an event
H4	Bounded / no-loss	footprint held near budget under overcommit; un-committed work checkpointed before any reclaim
H5	Overhead	continuous monitoring costs $\leq 2\%$ CPU at fleet scale

Table 1: The five runtime-verification signals and their pass thresholds.

Launcher	Attrib.	Cleanup	Escaped	Footprint _{N=10}	Ckpt.	Overhead
raw baseline	×	50%	×	2146 MiB	×	—
tmux 3.4	×	100%	×	2149 MiB	×	≈ 0
MEMMUX 0.8.0	✓	100%	✓ 10/10	435 MiB	✓ [†]	2.7%

Table 2: Head-to-head on the five signals (Linux reference host). MEMMUX is the only launcher that natively attributes per-agent memory (H1), surfaces escaped processes (H3), bounds the fleet’s footprint under overcommit (H4), and can checkpoint dirty work before a reclaim. It *matches* **tmux** on teardown cleanup (H2) and—stated plainly—pays a monitoring cost the ungoverned tools do not (H5). Attribution is the *native* capability, not the oracle’s coverage (§4). [†]Checkpoint-before-reclaim is confirmed by a daemon-level unit test; the pressure path was not triggered end-to-end in this benchmark (§5). Each cell is the headline result for that signal; the subsections give the detail.

160 with a live child, concurrently resident for the measurement window; without it, short
161 workloads finish before a fleet can be observed—an error we hit and corrected early.

162 **Claims discipline.** The harness records each launcher’s version, the measurement date,
163 and the host; it tags every sampled process as agent or manager overhead so the two are
164 reported separately; and it emits a number only for a monitor it actually measured. A
165 launcher that cannot be driven contributes “unsupported,” never a fabricated zero. These
166 rules are as much a contribution as the engine: fair comparison of agent-hosting tools has no
167 accepted methodology, and an unfair one is worse than none. One consequence deserves care.
168 Because the oracle can attribute *any* launcher’s tree after the fact, per-agent attribution is
169 *measurable* for all of them—the real-agent run reports 100% for every launcher (Table 4). H1
170 as a signal is the stricter question of whether a launcher exposes that attribution *natively*
171 *and continuously* to an operator, which only MEMMUX does; **tmux** and the raw baseline
172 surface no such figure themselves. It is that native capability, not the oracle’s coverage, that
173 Table 2 marks.

174 5 Evaluation

175 **Setup.** The reference host is an AWS m7i.2xlarge: 8 vCPUs (Intel Xeon Platinum
176 8488C), ≈ 31 GiB RAM, an 8 GiB swap file, Ubuntu 24.04 (Linux 6.17). Versions: **tmux**
177 3.4, MEMMUX 0.8.0. Each cell is two trials; we report mean $\pm 95\%$ CI. We swept fleet sizes
178 $N \in \{1, 5, 10\}$ across three scenarios. A macOS cross-check (Apple M4 Max, 32 GiB) that
179 also exercised **herdr** agreed qualitatively: like the other ungoverned managers, **herdr** reaped
180 its subtrees on teardown but saw no escapes and grew with the fleet. Table 2 summarises
181 the head-to-head; the subsections that follow are the evidence behind each cell.

Launcher	$N=1$	$N=5$	$N=10$
raw baseline	215	1074	2146
tmux 3.4	218	1076	2149
MEMMUX 0.8.0	220	435	435

Table 3: Peak fleet footprint (MiB) under a 3000 MiB budget (*overcommit*). The ungoverned launchers scale linearly with the fleet (raw and **tmux** track each other to within noise); MEMMUX holds flat at 435 MiB by deferring admissions—a $4.9\times$ separation at $N=10$.

182 5.1 H4: bounded footprint under overcommit

183 We give MEMMUX a budget of 3000 MiB—below the aggregate reservation of the larger
184 fleets—and start all N agents under each launcher. Table 3 reports peak total footprint
185 against fleet size. The ungoverned launchers scale linearly: at ten agents they reach 2.1 GB.
186 MEMMUX admits what fits (one agent at $N=1$; two at $N=5$ and $N=10$, deferring the
187 rest with a visible reason—3 and 8 deferrals respectively) and holds peak footprint flat at
188 435 MiB, a $4.9\times$ separation at $N=10$: without a budget, a fleet’s footprint is simply the
189 sum of its agents.

190 The *no-lost-work* half of H4 is a capability we verify but did not trigger here: because
191 MEMMUX defers at admission, the pressure ladder rarely needs to reclaim a running victim.
192 A daemon-level test confirms the guarantee directly—a task with a dirty Git worktree, forced
193 into the emergency stage, produces a checkpoint carrying its patch hash *before* the process
194 is terminated. Ungoverned launchers have no checkpoint mechanism, so this is reported as
195 a capability they lack, not a number.

196 5.2 H2: complete cleanup on teardown

197 Each launcher starts a fleet whose agents each hold a memory-bearing child, then is torn
198 down exactly as it normally would be (raw: kill the roots; **tmux**: **kill-server**; MEMMUX:
199 terminate each task). Ten seconds later we count survivors from the owned set. The raw
200 baseline strands every child it spawned—its teardown kills only the roots, orphaning the
201 grandchildren—and the leak grows with the fleet: one process and 30.6 MiB at $N=1$, ten
202 processes and 303 MiB at $N=10$ (50% reclaimed at every size). **tmux** and MEMMUX both
203 reap their full subtree at every size (0 leaked, 100% reclaimed). The honest reading, which
204 the claims discipline exists to surface: on *explicit* teardown, cleanup separates MEMMUX
205 from the raw baseline, not from a real multiplexer—**tmux** reaps its subtrees too.

206 5.3 H3: escaped-process visibility

207 We inject escapes: a stub forks an intermediate that spawns a memory-holding grandchild
208 and then exits, reparenting the grandchild to **init** while it stays alive. The harness inde-
209 pendently confirms the reparenting, then asks each launcher what it detected. MEMMUX
210 surfaces every escape—1/1, 5/5, 10/10—through events read from the daemon; the base-
211 lines have no mechanism to notice, so they are recorded as unsupported, never scored as
212 zero-of-measured. This is a capability, not a tuning difference, and it is the cleanest sepa-
213 ration from a real competitor. Detection is bounded by the sample interval: reconciliation
214 runs at 1 Hz, so a child that both escapes *and* exits within a second could slip between
215 two samples. Our injected escapes hold memory for several seconds—comfortably longer
216 than the interval—and shortening the interval trades detection latency for overhead (H5);
217 a persistent leak, which is the case that matters, is caught on the next sample regardless.

218 5.4 H1 & H5: what the attribution scan delivers, and what it costs

219 We pair these two signals because they are the benefit and the price of a single mecha-
220 nism: the always-on 1 Hz PSS scan produces the attribution guarantee (H1) and incurs
221 the monitoring cost (H5). Across every run MEMMUX attributed 100% of each launched
222 tree’s proportional memory to its owning task *natively*, and the external oracle indepen-
223 dently confirmed the figure (H1). The cost of governance in *memory* is small and we report

Table 4: Real Claude Code agents ($N=3$, hold scenario, macOS) driven through the identical harness. Every launcher attributes the full real (Node-based) agent tree; MEMMUX’s manager overhead is small. Footprint is steady-state provider+manager memory; at $N=3$ without an overcommit budget the totals are comparable across launchers—this run isolates *attribution and overhead on real trees*, not the overcommit footprint differentiator of Table 3.

Launcher	Total steady (MiB)	Manager (MiB)	Tree attributed
raw baseline	883.6	0.0	100%
tmux 3.6a	883.6	2.4	100%
herdr 0.8.0	929.6	18.2	100%
MEMMUX 0.8.0	843.5	7.2	100%

it plainly: the MEMMUX daemon’s own resident footprint is 4.6–5.4 MiB (growing slightly with the fleet), against 2.4 MiB for the tmux server and zero for the raw baseline. The cost in CPU is the honest weak spot (H5): the MEMMUX daemon’s total CPU—dominated by the always-on 1 Hz full-tree PSS scan, though we did not ablate the two—is 0.6% at a single agent, 1.5% at five, and 2.7% at ten, over our 2% target once the fleet reaches ten. The scan is $O(\text{processes})$ and currently re-reads the whole tree each tick; incremental sampling that touches only changed pids is the obvious fix and is future work. We state the miss rather than bury it.

5.5 Validation on real coding agents

To test whether the attribution and overhead results are artifacts of the stub, we ran the *same* harness against a real coding agent. Each “agent” is a headless Claude Code session (`claude -p`, v2.1.170) editing a scratch Git repository, launched through the identical sampling and teardown pipeline via the benchmark’s `--agent-cmd` mode. We report an $N=3$ hold run on the macOS host (Apple M4 Max, 36 GiB). A live Claude Code session is a multi-process, Node-based tree, so this is a strictly harder attribution target than the single-process stub.

Table 4 gives the result. MEMMUX attributes 100% of every real agent tree to its owning task—the H1 gate holds on live agents, not just the stub—and the manager cost stays small (7.2 MiB resident, 0.012% CPU at three agents, consistent with the small-fleet end of the H5 curve). Three concurrent real sessions occupy ~ 0.85 GB (~ 290 MiB each), confirming that the pressure the system governs is real and per-agent-substantial. We are deliberate about what this run does *not* show: because these sessions ran to completion and exited cleanly, every launcher (including the raw baseline) reclaimed 100% of its tree, so the cleanup and escaped-process *differentiators* do not appear here—they require the forced-termination and double-fork conditions that the controlled H2 and H3 scenarios above isolate. This experiment validates that attribution and overhead generalize from the stub to real, multi-process agent trees; the controlled scenarios remain the setting in which the governance differentiators are exercised.

6 Discussion and Limitations

Synthetic workload. The largest threat to validity: agents are a deterministic memory-ballast stub, not live models. We trade realism for reproducibility and for identical workloads across launchers, which is what makes the comparison fair. The *qualitative* guarantees—*attribution, cleanup, escape visibility, admission*—do not depend on the workload, but the specific megabytes for a real Claude- or GPT-driven session will differ. We take a first step in §5.5, running real Claude Code sessions through the same harness and confirming that 100% attribution and low overhead hold on live, multi-process agent trees; a larger live-agent study—more agents, longer sessions, and the forced-termination conditions that expose the cleanup differentiator—remains the most important next step.

One host, modest fleets. We evaluate a single 32 GB Linux host (with a macOS cross-check) and $N \leq 10$. The harness sweeps to $N=20$ and the reproducer emits the full curve;

we simply did not run larger fleets, on which the CPU-overhead question (H5) becomes sharper.

Conservative admission. MEMMUNIX reserves each task’s class-predicted peak, which for our stub is well above its actual RSS—so the flat 435 MiB in Table 3 reflects admission protecting against the *reservation*, and MEMMUNIX leaves headroom on the table (it deferred agents that would have fit). An exponentially-weighted predictor that tightens reservations toward observed peaks is designed and is future work. The bounded-footprint claim stands; the specific admit count would rise with a tighter predictor.

Overhead not yet met. Monitoring exceeds 2% CPU by ten agents (above); incremental sampling is the planned fix. As stated in §4, these are empirical monitors, not machine-checked proofs—checkable and checked, not formally verified.

7 Reproducibility

MEMMUNIX and its benchmark are open source (MIT).¹ A single command runs the full matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host specification, and regenerates every table and figure here plus the raw per-sample and per-process time series. Competitor versions and the measurement date are embedded in the generated report, and any launcher that could not be driven is listed with its reason. Reproducing the evaluation on another host is one invocation. The same harness accepts a `--agent-cmd` flag that substitutes a real coding-agent command (e.g. `claude -p ...`) for the stub, so the real-agent validation of §5.5 is reproducible through the identical pipeline; that run’s report and raw per-process series are archived under `paper/realagent-run/`.

```
memmux-bench paper --out paper-out          # full matrix
memmux-bench run  --agents 3 --agent-cmd \
'claude -p "..."' --out real-out            # same harness, real agent
```

Each run writes `host.json` (OS, CPU model, core count, physical RAM, tool versions), a Markdown report, and per-cell `*.jsonl` / `*.procs.jsonl` time series (RSS, PSS, USS, swap, and page faults per process on Linux), plus the figures.

8 Conclusion

Fleets of coding agents have outrun the tools we use to run them: multiplexers arrange them, but nothing governs them. We argued that memory governance for such fleets is a verification problem and built MEMMUNIX to make five guarantees checkable at runtime, with a benchmark honest enough to admit where those guarantees do *not* distinguish us. On a 32 GB Linux host the payoff is concrete—a flat versus linear footprint under overcommit (4.9× at ten agents), complete cleanup where the raw baseline strands half a fleet, and full visibility into escaped processes that the alternatives cannot see—alongside an honest overhead cost we have not yet driven under 2%. The path from here is a live-agent workload and larger fleets; the reproducer makes both a single command away. Verifying an agent is not only checking what it produced—it is also checking that the system running it kept its promises; the signals and the claims discipline here are a step toward the latter.

References

- [1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to run-time verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

¹Code, benchmark, and a one-command reproducer are open-source and will be released; the repository URL is withheld to preserve double-blind review.

- 311 [3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy
312 Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in
313 the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design
314 and Implementation (NSDI)*, 2011.
- 315 [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
316 Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In
317 *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- 318 [5] Linux Kernel Documentation. Control group v2. [https://www.kernel.org/doc/html/latest/
319 admin-guide/cgroup-v2.html](https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html), 2024. Accessed 2026-08.
- 320 [6] Linux Kernel Documentation. Out of memory management. [https://www.kernel.org/doc/
321 html/latest/admin-guide/mm/concepts.html](https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html), 2024. Accessed 2026-08.
- 322 [7] Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set
323 size). <https://www.kernel.org/doc/html/latest/filesystems/proc.html>, 2024. Accessed
324 2026-08.
- 325 [8] Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. [https://github.
326 com/tmux/tmux](https://github.com/tmux/tmux), 2024. Accessed 2026-08.
- 327 [9] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang,
328 and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In
329 *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology
330 (UIST)*, 2023. arXiv:2304.03442.
- 331 [10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao.
332 Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Infor-
333 mation Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- 334 [11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and
335 John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth
336 European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.
- 337 [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik
338 Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated soft-
339 ware engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- 340 [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan
341 Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference
342 on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.